

使用 ML Kit (Android) 偵測圖片中的物件

來源：<https://codelabs.developers.google.com/mlkit-android-odt?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將使用 ML Kit 建構 Android 應用程式，透過裝置端機器學習技術偵測圖片中的物件。

目錄

1. 事前準備
2. 做好準備
3. 將 ML Kit 物件偵測與追蹤 API 新增至專案
4. 執行範例應用程式
5. 新增裝置端物件偵測功能
6. 後續處理偵測結果
7. 恭喜！

步驟 1 · 約 0 分鐘

1. 事前準備

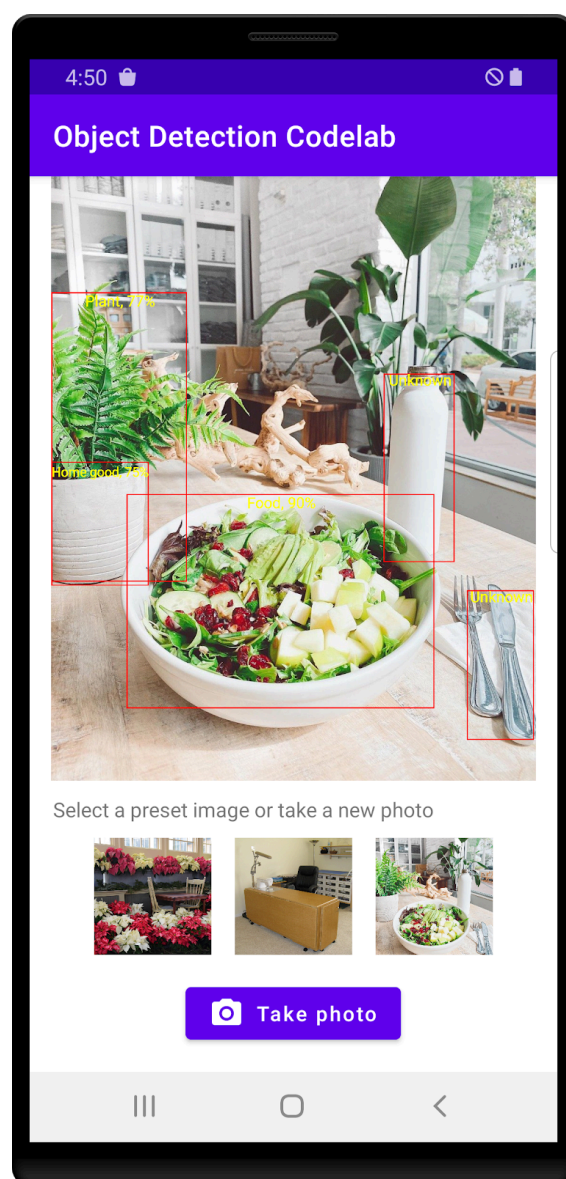
ML Kit 是一項行動 SDK，可將 Google 的裝置端機器學習專業技術導入 Android 和 iOS 應用程式。您可以運用功能強大但簡單易用的 Vision 和 Natural Language API，解決應用程式中的常見問題，或打造全新的使用者體驗。這些功能都採用 Google 頂尖的機器學習模型，而且完全免費。

ML Kit 的 API 全都在裝置端執行，因此適用於需要處理即時攝影機串流影像的即時應用情境。這也表示這項功能可離線使用。

本程式碼研究室將逐步說明如何將指定圖片的物體偵測和追蹤 (ODT) 功能新增至現有的 Android 應用程式。請注意，本程式碼研究室會採取一些捷徑，以突顯 ML Kit ODT 的用法。

建構項目

在本程式碼研究室中，您將使用 ML Kit 建構 Android 應用程式。您的應用程式會使用 ML Kit 物件偵測和追蹤 API，偵測指定圖片中的物件。最後，您應該會看到類似右側圖片的內容。



課程內容

- 如何將 ML Kit SDK 整合至 Android 應用程式
- ML Kit 物件偵測與追蹤 API

軟硬體需求

- 最新版 Android Studio (4.1.2 以上版本)
- Android Studio 模擬器或實體 Android 裝置
- [程式碼範例](#)
- 具備 Kotlin Android 開發作業的基本知識

本程式碼研究室著重於 ML Kit。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

2. 做好準備

下載程式碼

點選下方連結即可下載這個程式碼研究室的所有程式碼：

將下載的 ZIP 檔案解壓縮。這會解壓縮根資料夾 (`mlkit-android-main`)，其中包含您需要的所有資源。在本程式碼研究室中，您只需要 `object-detection` 子目錄中的來源。

mlkit-android 存放區中的物件偵測子目錄包含兩個目錄：

- **starter**：本程式碼研究室的範例程式碼，您將以這個程式碼為基礎進行建構。

- **final**：完成的範例應用程式程式碼。
-

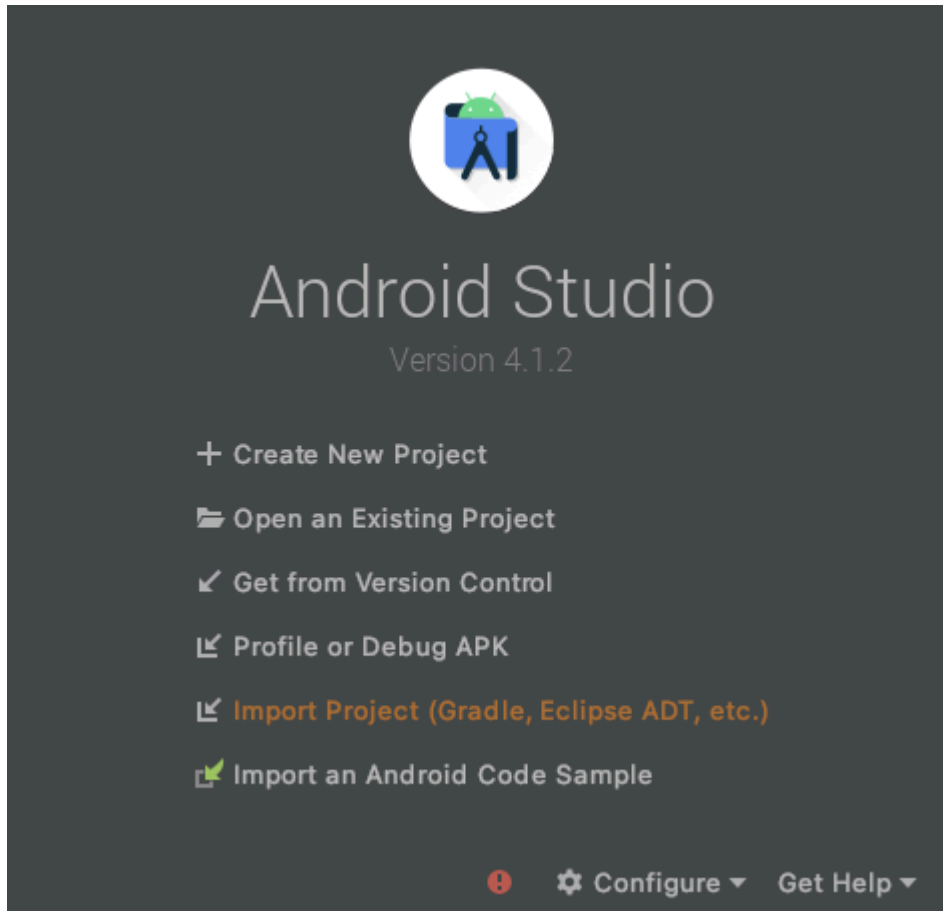
步驟 3 · 約 4 分鐘

3. 將 ML Kit 物件偵測與追蹤 API 新增至專案

將應用程式匯入 Android Studio

首先，請將範例應用程式匯入 Android Studio。

開啟 Android Studio，選取「Import Project (Gradle, Eclipse ADT, etc.)」，然後從先前下載的原始碼中選擇 `starter` 資料夾。



新增 ML Kit 物件偵測與追蹤的依附元件

ML Kit 依附元件可讓您在應用程式中整合 ML Kit ODT SDK。請在專案的 `app/build.gradle` 檔案結尾加入下列程式碼：

build.gradle

```
dependencies {  
    // ...  
    implementation 'com.google.mlkit:object-detection:16.2.4'  
}
```

將專案與 Gradle 檔案同步

為確保應用程式可使用所有依附元件，您應在此時將專案與 Gradle 檔案同步處理。

從 Android Studio 工具列選取「Sync Project with Gradle Files」（



)。

(如果這個按鈕已停用，請確認只匯入 `starter/app/build.gradle`，而非整個存放區)。

步驟 4 · 約 3 分鐘

4. 執行範例應用程式

您已將專案匯入 Android Studio，並新增 ML Kit 物件偵測和追蹤功能的依附元件，現在可以首次執行應用程式。

透過 USB 將 Android 裝置連線至主機，或[啟動 Android Studio 模擬器](#)，然後按一下 Android Studio 工具列中的「Run」(



)。

執行並探索應用程式

Android 裝置上應會啟動應用程式。其中包含一些樣板程式碼，可讓您擷取相片或選取預設圖片，並將其提供給您將在本程式碼研究室中建構的物件偵測和追蹤管道。編寫程式碼前，先來稍微瞭解一下這個應用程式。

首先，底部有一個按鈕 (

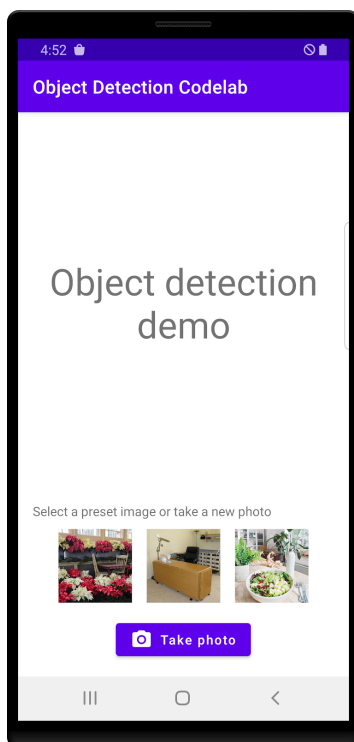


)，可執行下列操作：

- 啟動裝置/模擬器內建的相機應用程式
- 在**相機應用程式**中拍照
- 在**範例應用程式**中接收擷取的圖片
- 顯示圖片

試試「拍攝相片」按鈕，按照提示拍攝相片，然後**接受**相片，並觀察相片是否顯示在**範例應用程式**中。

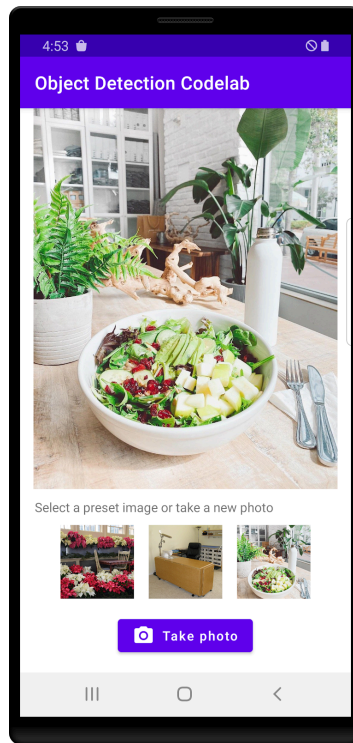
重複幾次，瞭解實際運作方式：





其次，你可以從 3 張預設圖片中選擇。如果您在 Android 模擬器上執行，稍後可以使用這些圖片測試物件偵測程式碼。

從 3 張預設圖片中選取圖片。確認圖片顯示在放大檢視畫面中：



步驟 5 · 約 5 分鐘

5. 新增裝置端物件偵測功能

在這個步驟中，您將在範例應用程式中新增功能，偵測圖片中的物件。如上一步所示，範例應用程式包含樣板程式碼，可透過裝置上的相機應用程式拍照。如果您在 Android Emulator 上執行程式碼研究室，也可以試用應用程式中的 3 張預設圖片進行物件偵測。

選取圖片後（無論是預設圖片或使用相機應用程式拍攝的相片），樣板程式碼會將該圖片解碼為 `Bitmap` 執行個體，並顯示在畫面上，然後使用圖片呼叫 `runObjectDetection` 方法。

在這個步驟中，您將在 `runObjectDetection` 方法中加入程式碼，執行物件偵測！

設定及執行圖片的裝置端物件偵測

設定 ML Kit ODT 只需要 3 個簡單步驟，並使用 3 個 API：

- 準備圖片：`InputImage`
- 建立偵測器物件：`ObjectDetection.getClient(options)`
- 連結上述 2 個物件：`process(image)`

您可以在 `MainActivity.kt` 檔案的 `runObjectDetection(bitmap: Bitmap)` 函式中完成這些作業。

```
/**
 * ML Kit Object Detection Function
 */
private fun runObjectDetection(bitmap: Bitmap) {
}
```

目前函式是空的。請繼續執行下列步驟，導入 *ML Kit ODT*！過程中，Android Studio 會提示您新增必要匯入項目：

- `com.google.mlkit.vision.common.InputImage`
- `com.google.mlkit.vision.objects.ObjectDetection`
- `com.google.mlkit.vision.objects.defaults.ObjectDetectorOptions`

步驟 1：建立 `InputImage`

ML Kit 提供簡單的 API，可從 `Bitmap` 建立 `InputImage`。接著，您就能將 `InputImage` 提供給 ML Kit API。

```
// Step 1: create ML Kit's InputImage object
val image = InputImage.fromBitmap(bitmap, 0)
```

將上述程式碼加到 `runObjectDetection(bitmap:Bitmap)` 頂端。

步驟 2：建立偵測工具執行個體

ML Kit 採用 *建構工具設計模式*。您會將設定傳遞至建構工具，然後從中取得 **detector**。您可以設定 3 個選項（本程式碼研究室會使用粗體選項）：

- 偵測器模式（**單一圖片**或串流）
- 偵測模式（**單一**或**多個**物件偵測）
- 分類模式（**開啟**或關閉）

本程式碼研究室適用於單一圖片 - 多個物件偵測和分類。立即新增：

```
// Step 2: acquire detector object
val options = ObjectDetectorOptions.Builder()
    .setDetectorMode(ObjectDetectorOptions.SINGLE_IMAGE_MODE)
    .enableMultipleObjects()
    .enableClassification()
    .build()
val objectDetector = ObjectDetection.getClient(options)
```

步驟 3：將圖片提供給偵測器

物件偵測和分類是異步處理：

- 將圖片傳送至偵測工具 (透過 `process()`)。
- 偵測器會盡力完成這項工作。
- 偵測器會透過回呼將結果回報給您。

下列程式碼會執行這項操作 (複製並附加至 `fun runObjectDetection(bitmap: Bitmap)`)：內現有的程式碼)：

```
// Step 3: feed given image to detector and setup callback
objectDetector.process(image)
    .addOnSuccessListener {
        // Task completed successfully
        debugPrint(it)
    }
    .addOnFailureListener {
        // Task failed with an exception
        Log.e(TAG, it.message.toString())
    }
```

完成後，偵測器會透過下列方式通知你：

- 偵測到的物件總數。系統會提供以下資訊，說明偵測到的每個物件：
- `trackingId`：用於跨影格追蹤的整數 (本程式碼研究室不會使用)。
- `boundingBox`：物件的定界框。
- `labels`：偵測到的物件標籤清單 (僅在啟用分類時)：
- `index` (取得這個標籤的索引)
- `text` (取得這個標籤的文字，包括「時尚商品」、「食品」、「居家用品」、「地點」、「植物」)
- `confidence` (介於 0.0 到 1.0 的浮點值，1.0 代表 100%)

您可能已注意到，程式碼會使用 `debugPrint()` 對偵測到的結果執行 `printf` 類型的處理作業。

將其新增至 `MainActivity` 類別：

```
private fun debugPrint(detectedObjects: List<DetectedObject>) {
    detectedObjects.forEachIndexed { index, detectedObject ->
        val box = detectedObject.boundingBox

        Log.d(TAG, "Detected object: $index")
        Log.d(TAG, " trackingId: ${detectedObject.trackingId}")
        Log.d(TAG, " boundingBox: (${box.left}, ${box.top}) - (${box.right},${box.bottom})")
        detectedObject.labels.forEach {
            Log.d(TAG, " categories: ${it.text}")
            Log.d(TAG, " confidence: ${it.confidence}")
        }
    }
}
```

現在您可以接受圖片進行偵測了！

按一下 Android Studio 工具列中的「Run」 (



)，執行程式碼研究室。請嘗試選取預設圖片或拍照，然後查看 IDE 內的 *logcat* 視窗(

 **Logcat**

)。

畫面應如下所示：

```
D/MLKit Object Detection: Detected object: 0
D/MLKit Object Detection: trackingId: null
D/MLKit Object Detection: boundingBox: (481, 2021) - (2426,3376)
D/MLKit Object Detection: categories: Food
D/MLKit Object Detection: confidence: 0.90234375
D/MLKit Object Detection: Detected object: 1
D/MLKit Object Detection: trackingId: null
D/MLKit Object Detection: boundingBox: (2639, 2633) - (3058,3577)
D/MLKit Object Detection: Detected object: 2
D/MLKit Object Detection: trackingId: null
D/MLKit Object Detection: boundingBox: (3, 1816) - (615,2597)
D/MLKit Object Detection: categories: Home good
D/MLKit Object Detection: confidence: 0.75390625
```

...這表示偵測器看到 3 個物件：

- 類別包括「食品」和「家居用品」。
- 由於第 2 個是未知類別，因此系統不會傳回任何類別。
- 無 `trackingId` (因為這是單一圖片偵測模式)。
- `boundingBox` 矩形內的位置 (例如 (481, 2021) - (2426, 3376))
- 偵測工具相當確定第 1 個是「食物」 (90% 信心度 - 是沙拉)。

從技術上來說，您目前已取得所有必要項目，可讓 *ML Kit* 物件偵測功能正常運作！恭喜！

在 UI 方面，您仍處於開始階段，但可以利用 UI 上偵測到的結果 (例如繪製出定界框)，打造更優質的體驗。讓我們前往下一個步驟，對偵測到的結果進行後續處理！

6. 後續處理偵測結果

在先前的步驟中，您將偵測到的結果列印到 *logcat*：簡單又快速。

在本節中，您將使用圖片中的結果：

- 在圖片上繪製定界框
- 在定界框內繪製類別名稱和信賴度

瞭解視覺化公用程式

程式碼研究室中提供一些樣板程式碼，協助您以視覺化方式呈現偵測結果。運用這些公用程式，簡化視覺化程式碼：

- `data class BoxWithText(val box: Rect, val text: String)` 這是用來儲存物體偵測結果以供視覺化的資料類別。`box` 是物件位置的定界框，`text` 則是與物件定界框一起顯示的偵測結果字串。
- `fun drawDetectionResult(bitmap: Bitmap, detectionResults: List<BoxWithText>): Bitmap` 這個方法會在輸入的 `bitmap` 上繪製 `detectionResults` 中的物件偵測結果，並傳回修改後的副本。

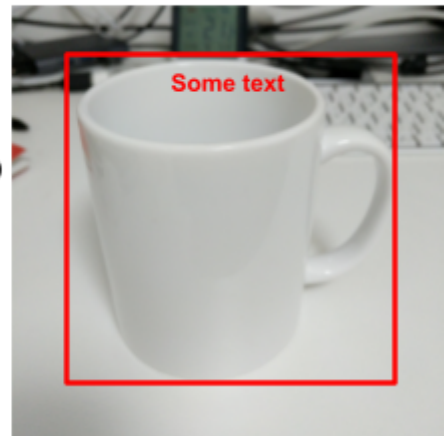
以下是 `drawDetectionResult` 公用程式方法的輸出範例：

```
detectionResults =  
[{"boundingBoxes": ((200,220), (1289, 1264)),  
  text: "Some text"}]
```

bitmap =



drawDetectionResult
(bitmap, detectionResults)



以視覺化方式呈現 ML Kit 偵測結果

使用視覺化公用程式，在輸入圖片上繪製 ML Kit 物件偵測結果。

前往呼叫 `debugPrint()` 的位置，並在下方新增下列程式碼片段：

```
// Parse ML Kit's DetectedObject and create corresponding visualization data
val detectedObjects = it.map { obj ->
    var text = "Unknown"

    // We will show the top confident detection result if it exist
    if (obj.labels.isNotEmpty()) {
        val firstLabel = obj.labels.first()
        text = "${firstLabel.text}, ${firstLabel.confidence.times(100).toInt()}%"
    }
    BoxWithText(obj.boundingBox, text)
}

// Draw the detection result on the input bitmap
val visualizedResult = drawDetectionResult(bitmap, detectedObjects)

// Show the detection result on the app screen
runOnUiThread {
    inputImageView.setImageBitmap(visualizedResult)
}
```

- 首先，請剖析 ML Kit 的 `DetectedObject`，並建立 `BoxWithText` 物件清單，以顯示視覺化結果。
- 接著，使用 `drawDetectionResult` 公用程式方法，在輸入圖片上繪製偵測結果，並顯示在畫面上。

開始執行

現在請按一下 Android Studio 工具列中的「Run」 (



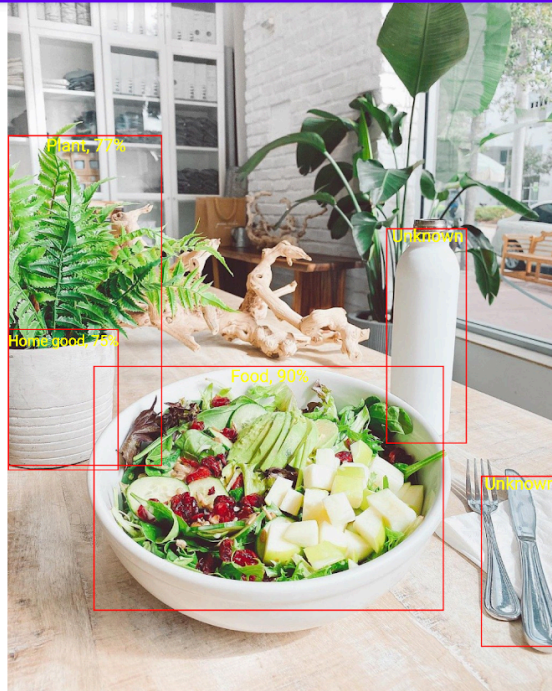
)。

應用程式載入後，按一下相機圖示按鈕，將相機對準物體，然後拍照、接受相片 (在「相機」應用程式中)，或輕觸任何預設圖片。您應該會看到偵測結果；再次按下按鈕或選取其他圖片，重複幾次，即可體驗最新的 ML Kit ODT！

4:50



Object Detection Codelab



Select a preset image or take a new photo



Take photo



步驟 7 · 約 0 分鐘

7. 恭喜！

您已使用 ML Kit 將物件偵測功能新增至應用程式：

- **3 個步驟和 3 個 API**
- 建立輸入圖片
- 建立偵測工具
- 將圖片傳送給偵測工具

這樣就能開始使用！

繼續操作時，您可能會想提升模型效能：如您所見，預設模型只能辨識 5 個類別，甚至不知道刀、叉和瓶子。如要瞭解如何訓練自訂模型，請參閱「裝置端機器學習 - 物件偵測」學習路徑中的其他程式碼研究室。

涵蓋內容

- 如何在 Android 應用程式中加入 ML Kit 物件偵測與追蹤功能
- 如何使用 ML Kit 的裝置端物件偵測和追蹤功能，偵測圖片中的物件

後續步驟

- 使用 ML Kit ODT 探索更多內容，透過更多圖片和即時影像體驗偵測和分類準確率及效能
- 請參閱「裝置端機器學習 - [物件偵測學習路徑](#)」，瞭解如何訓練自訂模型
- 在自己的 Android 應用程式中套用 ML Kit ODT

瞭解詳情

- <https://developers.google.com/ml-kit/vision/object-detection>
-